Class Test of Linked List

Name: Amnik Bose
Roll No - UG/04/BTCSE/2025/032
Reg. No - AU/2025/0000852

09/09/2025

1. b) Data and a pointer to the next node

2. c) NULL

3. b) Self-referential structure

4. b) head == NULL

5. b) newnode->next = head; head = newnode;

6. b) temp = head; head = head->next;
free(temp);

7. d) Deleting the head node

8. b) n

Amnik Bose ; ROLL.NO.- UG/04/BTCSE/2025/032 Reg No. - AU/2025/0000852

9. c) Copying the data of p->next into p then deleting p->next

10. c) while (p->data != NULL)

11. b) p->next == NULL

12. c) Two pointers advancing at different rates.

13. c) Three

14. b) Exactly one node

15. b) p = (struct node*) malloc(sizeof(struct node));

16. b) Avoid special-case handling for insertion and deletion at the head.

17. c) The first node

18. a) It cannot store duplicate values

Amnik Bose, ROLL NO. - UG/04/BTCSE/2025/032. Reg. No - AU/2025/0000852

19. b) Loss of access to all nodes, causing a memory leak

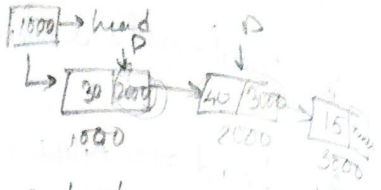
20. b) A slow pointer and a fast pointer

21. c) The last node

22. b) Nodes may be scattered anywhere in memory

23.

```
struct node {
    int data;
    struct node* next;
};
```



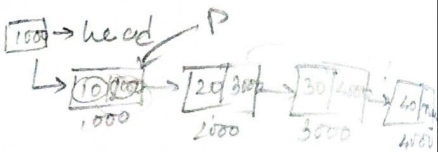

```
P = head
while (P->next != NULL)
    P = P->next
```

Amrita Bose, UG/04/BTCSE/2025/032, AU/2025/0000352

24. List : 30 → 50 → 70

25.

```
void insertEnd (struct node *head,
                int x)
{
struct node *t = (struct node*)malloc
        (sizeof (struct node));
t -> data = x;
t -> next = NULL;
struct node *P = head;
if (P != null)
{
while (P->next != NULL)
    {
    P = P->next;
P->next = t;
}
```




```
P = head;
P != NULL && P->next != NULL
Printf("%d", P->d
10 + 20
P = P->next
10+20=30
20+30=50
30+40=70
60+
=60
```

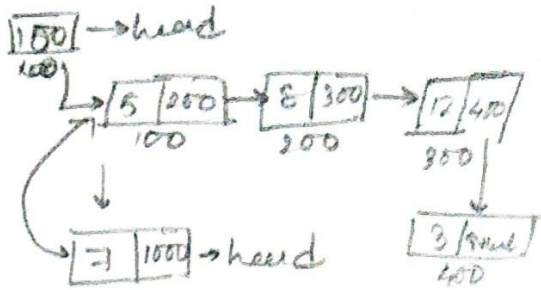
26.

```
struct node * reverseList (struct node *head) {
    struct node *prev = NULL;
    struct node *curr = head;
    struct node *next = NULL;
while (curr != NULL) {
    next = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = next;
}
return prev;
```

28.

a) Insert at beginning

```
insertatbeg(struct node *head, int info)
{
struct node *new;
new=(struct node*)malloc(sizeof(struct node));
new->data=info;
new->next=head;
head=new;
return head;
}
```




b) remove 12

```
delete_after(struct node *head, int key)
{
struct node *p1, *p2;
p1=head;
while(p1->next!=null)
{
if(p1->data==key)
{
p2=p1->next;
p1->next=p2->next;
free(p2);
break;
}
p1=p1->next;
}
return head;
}
```